Document Number: P2029R3
Date: 2020-09-11
Audience: CWG
Reply-to: Tom Honermann <tom@honermann.net>

Proposed resolution for core issues 411, 1656, and 2333; numeric and universal character escapes in character and string literals

Introduction

This paper proposes substantial changes to [lex.phases], [lex.ccon], and [lex.string] intended to address the following core issues as well as several other more minor issues.

- Core Issue 411: Use of universal-character-name in character versus string literals
- Core Issue 1656: Encoding of numerically-escaped characters
- Core Issue 2333: Escape sequences in UTF-8 character literals

This paper follows a prior proposed resolution that only attempted to address CWG 2333. That proposed resolution was discussed on the core mailing list and at the June 24th, 2019, core issues processing teleconference. The resolution proposed in this paper addresses the feedback provided during those discussions and further review at the January 16th, 2020, core issues processing teleconference, the March 23rd, 2020, core issues processing teleconference, discussion on the core mailing list originating on June 28th, 2020, further discussions on the core mailing list originating on July 7th, 2020, and the July 20th, 2020, core issues processing teleconference.

The notes for CWG 2333 in the current active issues list (revision 100) state that discussion at the August 14th, 2017, core issues processing teleconference resulted in a determination that numeric escape sequences in UTF-8 character literals should be ill-formed. The issue has remained in drafting status since then.

SG16 discussed this issue during its October 17th, 2018, teleconference. The SG16 consensus was for a different resolution than is currently described in the active issues list. The SG16 consensus was based on the following observations:

- The current resolution in the active issues list contradicts existing practice. gcc, Clang, and Visual C++ all allow octal and hexadecimal escape sequences in UTF-8 literals.
- Octal and hexadecimal escape sequences in UTF-8 literals are useful for a number of purposes:
  - Embedding null characters: `u8"\0"`
  - Creating ill-formed code unit sequences for testing purposes.
  - Creating Modified UTF-8, CESU-8, and WTF-8 string literals. This entails two abilities:
    - Embedding `U+0000` as an overlong UTF-8 sequence: `u8"\xC0\x80"`
    - Embedding lone surrogate code points as individual UTF-8 code unit sequences. For example, encoding `U+D800` as `u8"\xED\xA0\x80"`. (Note that use of `\u` escapes specifying surrogate code points is ill-formed).
  - Compatibility with existing log/debug systems that output literals with non-printable characters represented with escapes so as to facilitate copy/paste of such output into code.

SG16 conducted the following poll:
Continue to allow hex and octal escapes that indicate code unit values, requiring only that they fit into the range of the code unit type?

SF	F	N	A	SA
8	1	0	0	0

In the polled question, "Continue" refers to existing implementation behavior; to maintain the current implementation status quo exhibited by gcc, Clang and Visual C++.

The proposed resolution reflects the SG16 consensus.

CWG 411 is addressed by specifying different behavior for character literals vs string literals for characters that are not representable by a single code unit. For example, when the execution character set is UTF-8, `'\u0153'` is conditionally-supported, has type `int` and an implementation-defined value, but `"\u0153"` is a character array of length 3 containing the UTF-8 encoding of U+0153 (LATIN SMALL LIGATURE OE) and a null character (`\xC5\x93\x00`).

CWG 1656 is addressed by clarifying that numeric escape sequences denote code unit values in the execution character set; that the values are not subject to conversion from the encoding of the source file to the execution character set.

Additionally, since integer values are now guaranteed to be represented in two's complement following the adoption of P1236R1 in C++20, numeric escape sequences in ordinary and wide character and string literals that specify an integer value outside the representable range of the code unit type will now initialize the code unit similarly to an integral conversion if the integer value is within the range of representable values of the corresponding unsigned type of the code unit type, and are ill-formed otherwise. This matches the observed behavior in the gcc 10.1, Clang 10, Microsoft Visual C++ 19.24, and Intel 19 compilers. Previously such sequences resulted in implementation-defined behavior.

This paper does not attempt to address the existing issue concerning the order in which adjacent string literals are encoded and concatenated as reported at https://lists.isocpp.org/core/2020/07/9537.php and tracked by CWG 2455.

Changes from P2029R2

- Addressed changes requested during discussion on the core mailing list originating on July 7th, 2020:
  - [lex.ccon]pZ, [lex.string]pZ: Updated the wording to clarify that the "range of a [code unit] type" refers to the range of that type's representable values (and is not restricted only to code unit values that might appear in well-formed encoded text).
  - [lex.ccon]pZ, [lex.string]pZ: Updated wording for ordinary and wide character and string literals to specify that a numeric escape sequence that specifies an integer value outside the representable range of the code unit type will now initialize the code unit similarly to an integral conversion if the integer value is within the range of representable values of the corresponding unsigned type of the code unit type, and is ill-formed otherwise. Updated the introduction accordingly.
- [lex.ccon]pZ: Modified the order of Z.2-Z.4 to match the order of the equivalent presentation in [lex.string]pZ. This order presents the more common cases first.
- [lex.ccon]p6: Corrected the table number reference to [tab:lex.ccon.esc].
- Adressed changes requested during the July 20th, 2020, core issues processing teleconference:
  - Changed the introductory and proposed resolution overview to remove incorrect uses of "narrowing integer conversion" in prose intended to describe something more like an "integral conversion".
  - [lex.ccon]pY: Added footnotes explaining that, for nonencodable character literals and multicharacter literals, the associated character encoding may be used solely to determine encodability and not to actually encode values.
  - [lex.ccon]pZ: Merged Z.1 into the introductory text to make the short circuiting behavior clear and renumbered the remaining subparagraphs.
  - [lex.ccon]pZ: Specified that the determination of a character literal value during translation phase 4 uses the range of representable values of the character literal's type in translation phase 7.
  - [lex.string]pX: Added normative wording stating that the *n* that appears in the string literal array type corresponds to the number of encoded code units.
  - [lex.string]pZ: Elevated notes related to stateful character encodings to normative encouragement.

Proposed resolution overview

The proposed wording changes are intended to resolve CWG 411, CWG 1656, and CWG 2333 by:

- Clarifying that hexadecimal and octal escape sequences:
  - are valid in `u8`, `u`, and `U` character literals. (CWG 2333)
  - specify values that need not correspond to valid code unit values for the literal's associated character encoding. (CWG 2333)
  - specify the execution-time value of character literals or individual code unit values of string literals; that the value expressed is not subject to conversion to the literal's associated character encoding. (CWG 1656)
  - specify values that may result in string literals that are ill-formed according to the literal's associated character encoding. (CWG 2333)
- Clarifying that characters that cannot be specified in a character literal because they require more than one code unit in the literal's associated character encoding may be specified in string literals. (CWG 411)

Additionally, the wording updates are intended to:

- Specify behavior for non-ordinary character and string literals when a character lacks representation in the literal's associated character encoding (wording was previously missing).
- Broaden use of the term multicharacter literal to allow it to be applicable to both ordinary and wide character literals.
- Specify that wide multicharacter literals are conditionally supported (consistent with multicharacter literals).
- Specify that ordinary and wide string literals that specify characters that are not representable in the literal's associated character encoding are conditionally supported (gcc 10 rejects).
- Specify that wide character literals that have a `c-char-sequence` that specifies a character that is not representable in the execution wide-character set or that cannot be encoded in a single code unit are conditionally supported (consistent with ordinary character literals).
- Remove non-normative wording that states that `wchar_t` is able to represent all members of the execution wide-character set as this contradicts long standing existing practice for some implementations (Visual C++ has a 16-bit `wchar_t` type, but encodes wide strings as UTF-16; some characters require multiple code units).
- Modernize the wording with current style preferences and terminology.
- Move appending of a null character to string literals from translation phase 7 to translation phase 6.
- Specify that a numeric escape sequence in ordinary and wide character and string literals that specifies an integer value outside the representable range of the code unit type will now initialize the code unit similarly to an integral conversion if the integer value is within the range of representable values of the corresponding unsigned type of the code unit type, and is ill-formed otherwise.

The wording changes introduce the following:

- The concept of an "associated character encoding" for character and string literals; this enables wording to be independent of the particular kind of literal (ordinary, wide or Unicode).
- New `basic-c-char`, `basic-s-char`, `numeric-escape-sequence`, and `simple-escape-sequence-char` grammar non-terminals; these factor the existing `c-char`, `s-char`, `escape-sequence`, and `simple-escape-sequence` non-terminals to give names to their productions for reference purposes.
- New `conditional-escape-sequence` and `conditional-escape-sequence-char` grammar non-terminals; these address previously missing grammar support for conditionally-supported implementation-defined escape sequences.
- A new `non-encodable character literal` kind; this provides a name for conditionally-supported implementation-defined ordinary and wide character literals that have `c-char-sequences` that specify characters that either lack representation in the literal's associated character encoding or that cannot be encoded as a single code unit.

Implementation impact

The author intends the proposed wording to reflect existing practice for the gcc and Clang compilers. If this proposal is adopted, neither of those compilers are expected to require updates. However, there is implementation impact to the Microsoft Visual C++ compiler.

Semantics of numeric-escape-sequences in UTF-8 literals

Consider the following (C++20) code:

```
constexpr const char8_t c[] = u8"\xc3\x80"; // UTF-8 encoding of U+00C0 {LATIN CAPITAL LETTER A WITH GRAVE}
#if defined(_MSC_VER)
  // Microsoft Visual C++:
```

```
  static_assert(c[0] == 0xC3); // UTF-8 encoding of U+00C3 {LATIN CAPITAL LETTER A WITH TILDE}
  static_assert(c[1] == 0x83);
  static_assert(c[2] == 0xC2); // UTF-8 encoding of U+0080 {<control>}
  static_assert(c[3] == 0x80);
  static_assert(c[4] == 0x00); // null
#else
  // Gcc and Clang:
  static_assert(c[0] == 0xC3); // UTF-8 encoding of U+00C0 {LATIN CAPITAL LETTER A WITH GRAVE}
  static_assert(c[1] == 0x80);
  static_assert(c[2] == 0x00); // null
#endif
```

Gcc and Clang encode the hexadecimal escapes as code units in the target (UTF-8) encoding and perform no conversions (consistent with the behavior intended by this proposal). However, Visual C++ considers each hexadecimal escape to specify a code point in the source encoding and encodes each as UTF-8.

The author does not know if the Visual C++ behavior exhibited for UTF-8 literals is intentional or reflective of a defect. The behavior is inconsistent for UTF-8 and UTF-16 literals. For UTF-8 literals, numeric escape sequences that specify values outside the range of `char8_t` are accepted as code point values and encoded as UTF-8. However, for UTF-16 literals, numeric escape sequences that specify values outside the range of `char16_t` are rejected rather than being considered a code point value and encoded as a UTF-16 surrogate pair.

Character literal type for characters not representable in the execution character set

Consider the following code assuming that the execution character set does not have representation for the specified Unicode code points.

```
auto c1 = '\uFF10';
extern char c1;
#ifdef _MSC_VER
static_assert('\uFF10' == '?');
#endif
auto c2 = '\U0001F235';
extern int c2;
#ifdef _MSC_VER
static_assert('\U0001F235' == '??');
#endif
```

This code should be rejected (both before and after this proposal) because the redeclaration of `c1` with type `char` does not match the first declaration for which `c1` should have a deduced type of `int`. Visual C++ accepts it when compiled with `/execution-charset:windows-1252` with the following warnings:

```
<source>(1): warning C4566: character represented by universal-character-name '\uFF10' cannot be represented in the current code page (1252)
<source>(4): warning C4566: character represented by universal-character-name '\uFF10' cannot be represented in the current code page (1252)
<source>(6): warning C4566: character represented by universal-character-name '\U0001F235' cannot be represented in the current code page (1
<source>(9): warning C4566: character represented by universal-character-name '\U0001F235' cannot be represented in the current code page (1
```

It seems that the Visual C++ compiler translates unrepresentable characters from the Unicode BMP to a single `char` with value equal to `'?'`, but translates unrepresentable characters from outside the Unicode BMP to `int` with value equal to the multicharacter literal `'??'`. This seems unlikely to be intended behavior. It would be conforming if, for the Unicode BMP case, an `int` with value equal to `'?'` was produced.

Gcc and Clang both reject the above code regardless of whether those Unicode characters have representation in the execution character set. If they are representable, then the code is rejected (as permitted) because the characters cannot be encoded in a single code unit. If they are not representable (which only happens for gcc since Clang always targets UTF-8), then the code is rejected because the redeclaration of `c1` as `char` does not match the deduced `int` type for its first declaration.

Acknowledgements

Thank you to Steve Downey, Corentin Jabot, Jens Maurer, Alisdair Meredith, Richard Smith, and Hubert Tong for their tireless feedback on numerous drafts of this paper!

Proposed resolution

These changes are relative to N4861.

The changes to [lex.ccon] and [lex.string] are rather pervasive. For ease of review, unchanged paragraphs in these sections are retained in the wording below. These paragraphs are introduced with "*No* changes to ..." and are highlighted with a blue background.

- [ ] Hide inserted text
- [ ] Hide deleted text

[lex.phases]

Change in 5.2 [lex.phases] paragraph 1.5:

> Each ~~basic source character set member~~*basic-c-char*, *basic-s-char*, and *r-char* in a *character-literal* or a *string-literal*, as well as each ~~escape sequence~~*escape-sequence* and *universal-character-name* in a *character-literal* or a non-raw string literal, ~~is converted to the corresponding member of the execution character set ([lex.ccon], [lex.string]);~~ is encoded in the literal's associated character encoding as specified in [lex.ccon] and [lex.string]. ~~if there is no corresponding member, it is converted to an implementation-defined member other than the null (wide) character.~~ ~~[Footnote: An implementation need not convert all non-corresponding source characters to the same execution character.]~~

Change in 5.2 [lex.phases] paragraph 1.6:
Drafting note: This addition duplicates wording in [lex.string], but seems important to include here.

> Adjacent ~~string literal tokens~~*string-literals* are concatenated and a null character is appended to each *string-literal* as specified in [lex.string].

[lex.ccon]

Change in 5.13.3 [lex.ccon]:

character-literal:
encoding-prefix$_{opt}$ ' c-char-sequence '

encoding-prefix: one of
`u8 u U L`

c-char-sequence:
c-char
c-char-sequence c-char

c-char:
~~any member of the basic source character set except the single-quote ', backslash \, or new-line character~~
basic-c-char
escape-sequence
universal-character-name

basic-c-char:
any member of the basic source character set except the single-quote ', backslash \, or new-line character

escape-sequence:
simple-escape-sequence
~~octal-escape-sequence~~
~~hexadecimal-escape-sequence~~
numeric-escape-sequence
conditional-escape-sequence

simple-escape-sequence: ~~one of~~
~~`\' \" \? \\`~~
~~`\a \b \f \n \r \t \v`~~
\ simple-escape-sequence-char

simple-escape-sequence-char: one of
`' " ? \`
`a b f n r t v`

numeric-escape-sequence:
octal-escape-sequence
hexadecimal-escape-sequence

octal-escape-sequence:
\ octal-digit
\ octal-digit octal-digit
\ octal-digit octal-digit octal-digit

hexadecimal-escape-sequence:
`\x` hexadecimal-digit
hexadecimal-escape-sequence hexadecimal-digit

conditional-escape-sequence:
\ conditional-escape-sequence-char

conditional-escape-sequence-char:
any member of the basic source character set that is not an octal-digit, a simple-escape-sequence-char, or the characters `u`, `U`, or `x`

Delete 5.13.3 [lex.ccon] paragraph 1:
Drafting Note: The contents of paragraphs 1-5 were incorporated into new paragraphs.

~~A *character-literal* that does not begin with `u8`, `u`, `U`, or `L` is an *ordinary character literal*. An ordinary character literal that contains a single *c-char* representable in the execution character set has type `char`, with value equal to the numerical value of the encoding of the *c-char* in the execution character set. An ordinary character literal that contains more than one *c-char* is a *multicharacter literal*. A multicharacter literal, or an ordinary character literal containing a single *c-char* not representable in the execution character set, is conditionally-supported, has type `int`, and has an implementation-defined value.~~

Delete 5.13.3 [lex.ccon] paragraph 2:
Drafting Note: The contents of paragraphs 1-5 were incorporated into new paragraphs. The note regarding the range of single code unit values was removed.

~~A *character-literal* that begins with `u8`, such as `u8'w'`, is a *character-literal* of type `char8_t`, known as a *UTF-8 character literal*. The value of a UTF-8 character literal is equal to its ISO/IEC 10646 code point value, provided that the code point value can be encoded as a single UTF-8 code unit. [*Note:* That is, provided the code point value is in the range [0,7F] (hexadecimal). — *end note*] If the value is not representable with a single UTF-8 code unit, the program is ill-formed. A UTF-8 character literal containing multiple *c-char*s is ill-formed.~~

Delete 5.13.3 [lex.ccon] paragraph 3:
Drafting Note: The contents of paragraphs 1-5 were incorporated into new paragraphs. The note regarding the range of singled code unit values was removed.

~~A *character-literal* that begins with the letter `u`, such as `u'x'`, is a *character-literal* of type `char16_t`, known as a *UTF-16 character literal*. The value of a UTF-16 character literal is equal to its ISO/IEC 10646 code point value, provided that the code point value is representable with a single 16-bit code unit. [*Note:* That is, provided the code point value is in the range [0,FFFF] (hexadecimal). — *end note*] If the value is not representable with a single 16-bit code unit, the program is ill-formed. A UTF-16 character literal containing multiple *c-char*s is ill-formed.~~

Delete 5.13.3 [lex.ccon] paragraph 4:
Drafting Note: The contents of paragraphs 1-5 were incorporated into new paragraphs.

~~A *character-literal* that begins with `U`, such as `U'y'`, is a *character-literal* of type `char32_t`, known as a *UTF-32 character literal*. The value of a UTF-32 character literal containing a single *c-char* is equal to its ISO/IEC 10646 code point value. A UTF-32 character literal containing multiple *c-char*s is ill-formed.~~

Delete 5.13.3 [lex.ccon] paragraph 5:
Drafting Note: The contents of paragraphs 1-5 were incorporated into new paragraphs. The note regarding the ability for `wchar_t` to store all values of the execution wide-character set is intentionally removed as it conflicts with long standing existing practice for some implementations.

~~A *character-literal* that begins with the letter `L`, such as `L'z'`, is a *wide-character literal*. A wide-character literal has type `wchar_t`.~~ ~~[Footnote: They are intended for character sets where a character does not fit into a single byte.]~~ ~~The value of a wide-character literal containing a single *c-char* has value equal to the numerical value of the encoding of the *c-char* in the execution wide-character set, unless the *c-char* has no representation in the execution wide-character set, in which case the value is implementation-defined. [*Note:* The type `wchar_t` is able to represent all members of the execution wide-character set (see [basic.fundamental]). — *end note*] The value of a wide-character literal containing multiple *c-char*s is implementation-defined.~~

Add a new paragraph (W) before 5.13.3 [lex.ccon] paragraph 6:

A *non-encodable character literal* is a *character-literal* whose *c-char-sequence* consists of a single *c-char* that is not a *numeric-escape-sequence* and that specifies a character that either lacks representation in the literal's associated character encoding or that cannot be encoded as a single code unit. A *multicharacter literal* is a *character-literal* whose *c-char-sequence* consists of more than one *c-char*. The *encoding-prefix* of a non-encodable character literal or a multicharacter literal shall be absent or `L`. Such *character-literal*s are conditionally-supported.

Add another new paragraph (X) and table (Y) before 5.13.3 [lex.ccon] paragraph 6:

The kind of a *character-literal*, its type, and its associated character encoding are determined by its *encoding-prefix* and its *c-char-sequence* as defined by table Y. The special cases for non-encodable character literals and multicharacter literals take precedence over their respective base kinds.

Table Y: Character literals [tab:lex.ccon.literals]

Encoding prefix	Kind	Type	Associated character encoding	Example
none	*ordinary character literal*	`char`	encoding of the execution character set [Footnote Y1: *see below*]	`'v'`
	— non-encodable ordinary character literal	`int`		`'\U0001F525'` [Footnote Y2: *see below*]
	— ordinary multicharacter literal			`'abcd'`
`L`	*wide character literal*	`wchar_t`	encoding of the execution wide-character set [Footnote Y3: *see below*]	`L'w'`
	— non-encodable wide character literal			`L'\U0001F32A'` [Footnote Y4: *see below*]
	— wide multicharacter literal			`L'abcd'`
`u8`	*UTF-8 character literal*	`char8_t`	UTF-8	`u8'x'`
`u`	*UTF-16 character literal*	`char16_t`	UTF-16	`u'y'`
`U`	*UTF-32 character literal*	`char32_t`	UTF-32	`U'z'`

Y1) The associated character encoding for ordinary character literals determines encodability, but does not determine the value of non-encodable ordinary character literals or ordinary multicharacter literals.
Y2) The example assumes that the specified character lacks representation in the execution character set, or that encoding it would require more than one code unit.
Y3) The associated character encoding for wide character literals determines encodability, but does not determine the value of non-encodable wide character literals or wide multicharacter literals.
Y4) The example assumes that the specified character lacks representation in the execution wide-character set, or that encoding it would require more than one code unit.

Add another new paragraph (Z) before 5.13.3 [lex.ccon] paragraph 6:

In translation phase 4, the value of a *character-literal* is determined using the range of representable values of the *character-literal*'s type in translation phase 7. A non-encodable character literal or a multicharacter literal has an implementation-defined value. The value of any other kind of *character-literal* is determined as follows:

(Z.1) — A *character-literal* with a *c-char-sequence* consisting of a single *basic-c-char*, *simple-escape-sequence*, or *universal-character-name* is the code unit value of the specified character as encoded in the literal's associated character encoding. [*Note:* If the specified character lacks representation in the literal's associated character encoding or if it cannot be encoded as a single code unit, then the literal is a non-encodable character literal. — *end note*]

(Z.2) — A *character-literal* with a *c-char-sequence* consisting of a single *numeric-escape-sequence* that specifies an integer value `V` has a value as follows:
- — If `V` does not exceed the range of representable values of the *character-literal*'s type, then the value is `V`.
- — Otherwise, if the *character-literal*'s *encoding-prefix* is absent or `L`, and `V` does not exceed the range of representable values of the corresponding unsigned type for the underlying type of the *character-literal*'s type, then the value is the unique value of the *character-literal*'s type `T` that is congruent to `V` modulo 2^N, where N is the width of `T`.
- — Otherwise, the *character-literal* is ill-formed.

(Z.3) — A *character-literal* with a *c-char-sequence* consisting of a single *conditional-escape-sequence* is conditionally-supported and has an implementation-defined value.

Change in 5.13.3 [lex.ccon] paragraph 6:
Drafting Note: The deleted text has been removed as redundant since it repeats information implicit in the grammar. The added note is content moved from the deleted footnote.

The character specified by a *simple-escape-sequence* is specified in table 9. [*Note:* Using an escape sequence for a question mark is supported for compatibility with ISO C++ 2014 and ISO C. — *end note*] ~~Certain non-graphic characters, the single quote `'`, the double quote `"`, the question mark `?`, [Footnote: Using an escape sequence for a question mark is supported for compatibility with ISO C++ 2014 and ISO C.]~~ and the backslash `\`, can be represented according to Table 9. ~~The double quote `"` and the question mark `?`, can be represented as themselves or by the escape sequences `\"` and `\?` respectively, but the single quote `'` and the backslash `\` shall be represented by the escape sequences `\'` and `\\` respectively. Escape sequences in which the character following the backslash is not listed in Table 9 are conditionally-supported, with implementation-defined semantics. An escape sequence specifies a single character.~~

Table 9: ~~E~~Simple escape sequences [tab:lex.ccon.esc]

new-line	NL(LF)	`\n`
horizontal tab	HT	`\t`
vertical tab	VT	`\v`
backspace	BS	`\b`
carriage return	CR	`\r`
form feed	FF	`\f`
alert	BEL	`\a`
backslash	\	`\\`
question mark	?	`\?`
single quote	'	`\'`
double quote	"	`\"`
~~octal number~~	~~ooo~~	~~`\ooo`~~
~~hex number~~	~~hhh~~	~~`\xhhh`~~

Delete 5.13.3 [lex.ccon] paragraph 7:
Drafting Note: Wording describing the form of octal and hexadecimal escape sequences has been removed as redundant; the form is implicit in the grammar.

~~The escape `\ooo` consists of the backslash followed by one, two, or three octal digits that are taken to specify the value of the desired character. The escape `\xhhh` consists of the backslash followed by `x` followed by one or more hexadecimal digits that are taken to specify the value of the desired character. There is no limit to the number of digits in a hexadecimal sequence. A sequence of octal or hexadecimal digits is terminated by the first character that is not an octal digit or a hexadecimal digit, respectively. The value of a *character-literal* is implementation-defined if it falls outside of the implementation-defined range defined for `char` (for *character-literal*s with no prefix) or `wchar_t` (for *character-literal*s prefixed by `L`). [*Note:* If the value of a *character-literal* prefixed by `u`, `U8`, or `U` is outside the range defined for its type, the program is ill-formed. — *end note*]~~

Delete 5.13.3 [lex.ccon] paragraph 8:
Drafting Note: The normative text was combined with wording for `basic-c-char` and `simple-escape-sequence` above. The deleted note duplicates normative text in 5.2 [lex.phases] paragraph 1.1.

~~A *universal-character-name* is translated to the encoding, in the appropriate execution character set, of the character named. If there is no such encoding, the *universal-character-name* is translated to an implementation-defined encoding. [*Note:* In translation phase 1, a *universal-character-name* is introduced whenever an actual extended character is encountered in the source text. Therefore, all extended characters are described in terms of *universal-character-name*s However, the actual compiler implementation may use its own native character set, so long as the same results are obtained. — *end note*]~~

[lex.string]

Change in 5.13.5 [lex.string]:

string-literal:
  encoding-prefix$_{opt}$ " s-char-sequence$_{opt}$ "
  encoding-prefix$_{opt}$ `R` raw-string

s-char-sequence:
  s-char
  s-char-sequence s-char

s-char:
  ~~any member of the basic source character set except the double-quote `"`, backslash `\`, or new-line character~~
  basic-s-char
  escape-sequence
  universal-character-name

basic-s-char:
  any member of the basic source character set except the double-quote `"`, backslash `\`, or new-line character

raw-string:
  " d-char-sequence$_{opt}$ `(` r-char-sequence$_{opt}$ `)` d-char-sequence$_{opt}$ "

r-char-sequence:

r-char
r-char-sequence r-char

r-char:
any member of the source character set, except a right parenthesis `)` followed by
the initial d-char-sequence (which may be empty) followed by a double quote `"`.

d-char-sequence:
d-char
d-char-sequence d-char

d-char:
any member of the source character set except:
space, the left parenthesis `(`, the right parenthesis `)`, the backslash `\`, and the control characters
representing horizontal tab, vertical tab, form feed, and newline.

Add a new paragraph (X) and table (Y) before 5.13.5 [lex.string] paragraph 1:

The kind of a *string-literal*, its type, and its associated character encoding are determined by its encoding prefix and sequence of *s-char*s or *r-char*s as defined by table Y where *n* is the number of encoded code units as described below.

Table Y: String literals [tab:lex.string.literals]

Encoding prefix	Kind	Type	Associated character encoding	Examples
none	*ordinary string literal*	array of *n* `const char`	encoding of the execution character set	`"an ordinary string"` `R"(an ordinary raw string)"`
`L`	*wide string literal*	array of *n* `const wchar_t`	encoding of the execution wide-character set	`L"a wide string"` `LR"w(a wide raw string)w"`
`u8`	*UTF-8 string literal*	array of *n* `const char8_t`	UTF-8	`u8"a UTF-8 string"` `u8R"x(a UTF-8 raw string)x"`
`u`	*UTF-16 string literal*	array of *n* `const char16_t`	UTF-16	`u"a UTF-16 string"` `uR"y(a UTF-16 raw string)y"`
`U`	*UTF-32 string literal*	array of *n* `const char32_t`	UTF-32	`U"A UTF-32 string"` `UR"z(a UTF-32 raw string)z"`

No changes to 5.13.5 [lex.string] paragraph 1:

A *string-literal* that has an `R` in the prefix is a *raw string literal*. The *d-char-sequence* serves as a delimiter. The terminating *d-char-sequence* of a *raw-string* is the same sequence of characters as the initial *d-char-sequence*. A *d-char-sequence* shall consist of at most 16 characters.

No changes to 5.13.5 [lex.string] paragraph 2:

[*Note:* The characters `'('` and `')'` are permitted in a *raw-string*. Thus, `R"delimiter((a|b))delimiter"` is equivalent to `"(a|b)"`. — *end note*]

No changes to 5.13.5 [lex.string] paragraph 3:

[*Note:* A source-file new-line in a raw string literal results in a new-line in the resulting execution string literal. Assuming no whitespace at the beginning of lines in the following example, the assert will succeed:

```
const char* p = R"(a\
b
c)";
assert(std::strcmp(p, "a\\\nb\nc") == 0);
```

— *end note*]

No changes to 5.13.5 [lex.string] paragraph 4:

[*Example:* The raw string

```
R"a(
)\
a"
)a"
```

is equivalent to `"\n)\\\na\"\n"`. The raw string

```
R"(x = "\"y\"")"
```

is equivalent to `"x = \"\\\"y\\\"\""`. — *end example*]

Delete 5.13.5 [lex.string] paragraph 5:
Drafting Note: The contents of paragraphs 5, 6, 8, 9, and 10 were incorporated into new paragraphs.

~~After translation phase 6, a *string-literal* that does not begin with an *encoding-prefix* is an *ordinary string literal*. An ordinary string literal has type "array of *n* const char" where *n* is the size of the string as defined below, has static storage duration ([basic.stc]), and is initialized with the given characters.~~

Delete 5.13.5 [lex.string] paragraph 6:
Drafting Note: The contents of paragraphs 5, 6, 8, 9, and 10 were incorporated into new paragraphs.

A *string-literal* that begins with `u8`, such as `u8"asdf"`, is a *UTF-8 string literal*. A UTF-8 string literal has type "array of *n* const char8_t", where *n* is the size of the string as defined below; each successive element of the object representation ([basic.types]) has the value of the corresponding code unit of the UTF-8 encoding of the string.

No changes to 5.13.5 [lex.string] paragraph 7:

Ordinary string literals and UTF-8 string literals are also referred to as narrow string literals.

Delete 5.13.5 [lex.string] paragraph 8:
Drafting Note: The contents of paragraphs 5, 6, 8, 9, and 10 were incorporated into new paragraphs. The note has been deleted as redundant; the use of surrogate pairs is explicit in the UTF-16 encoding.

A *string-literal* that begins with `u`, such as `u"asdf"`, is a *UTF-16 string literal*. A UTF-16 string literal has type "array of *n* const char16_t", where *n* is the size of the string as defined below; each successive element of the array has the value of the corresponding code unit of the UTF-16 encoding of the string. [*Note:* A single *c-char* may produce more than one char16_t character in the form of surrogate pairs. A surrogate pair is a representation for a single code point as a sequence of two 16-bit code units. — *end note*]

Delete 5.13.5 [lex.string] paragraph 9:
Drafting Note: The contents of paragraphs 5, 6, 8, 9, and 10 were incorporated into new paragraphs.

A *string-literal* that begins with `U`, such as `U"asdf"`, is a *UTF-32 string literal*. A UTF-32 string literal has type "array of *n* const char32_t", where *n* is the size of the string as defined below; each successive element of the array has the value of the corresponding code unit of the UTF-32 encoding of the string.

Delete 5.13.5 [lex.string] paragraph 10:
Drafting Note: The contents of paragraphs 5, 6, 8, 9, and 10 were incorporated into new paragraphs.

A *string-literal* that begins with `L`, such as `L"asdf"`, is a *wide string literal*. A wide string literal has type "array of *n* const wchar_t", where *n* is the size of the string as defined below; it is initialized with the given characters.

Change in 5.13.5 [lex.string] paragraph 11:

In translation phase 6 ([lex.phases]), adjacent *string-literal*s are concatenated. If both *string-literal*s have the same *encoding-prefix*, the resulting concatenated *string-literal* has that *encoding-prefix*. If one *string-literal* has no *encoding-prefix*, it is treated as a *string-literal* of the same *encoding-prefix* as the other operand. If a UTF-8 string literal token is adjacent to a wide string literal token, the program is ill-formed. Any other concatenations are conditionally-supported with implementation-defined behavior. [*Note:* This concatenation is an interpretation, not a conversion. Because the interpretation happens in translation phase 6 (after each character from a *string-literal* has been translated into a value from the appropriate character setafter the string literal contents have been encoded in the *string-literal*'s associated character encoding), a *string-literal*'s initial rawness has no effect on the interpretation or well-formedness of the concatenation. — *end note*] Table 11 has some examples of valid concatenations.

Table 11: String literal concatenations [tab:lex.string.concat]

Source	Means	Source	Means	Source	Means
`u"a" u"b"`	`u"ab"`	`U"a" U"b"`	`U"ab"`	`L"a" L"b"`	`L"ab"`
`u"a" "b"`	`u"ab"`	`U"a" "b"`	`U"ab"`	`L"a" "b"`	`L"ab"`
`"a" u"b"`	`u"ab"`	`"a" U"b"`	`U"ab"`	`"a" L"b"`	`L"ab"`

Characters in concatenated strings are kept distinct.

[*Example:*

```
"\xA" "B"
```

contains the two characters `'\xA'` and `'B'` after concatenation (and not the single hexadecimal character `'\xAB'`). — *end example*]

Change in 5.13.5 [lex.string] paragraph 12:
Drafting note: The literal `'\0'` was replaced with "a null character" to avoid an otherwise needed correction to qualify the literal to be appended with an encoding prefix matching the kind of the string literal. The rationale regarding why a null character is appended was removed as unnecessary normative text. If desired, it could be restored as a note.

After any necessary concatenation, iIn translation phase 76 ([lex.phases]), after adjacent *string-literal*s are concatenated, `'\0'`a null character is appended to every *string-literal* so that programs that scan a string can find its end.

Delete 5.13.5 [lex.string] paragraph 13:
Drafting note: This wording has been removed as misleading, incomplete, or redundant. String literal contents do not always have the same meaning as in character literals. The wording regarding single and double quotes is redundant with the grammar. The discussion of string length is unnecessary as string length is determined by encoding.

Escape sequences and *universal-character-name*s in non-raw string literals have the same meaning as in character literals ([lex.ccon]), except that the single quote `'` is representable either by itself or by the escape sequence `\'`, and the double quote `"` shall be preceded by a `\`, and except that a *universal-character-name* in a UTF-16 string literal may yield a surrogate pair. In a narrow string literal, a *universal-character-name* may map to more than one `char` or `char8_t` element due to *multibyte encoding*. The size of a `char32_t` or wide string literal is the total number of escape sequences, *universal-character-name*s, and other characters, plus one for the terminating `U'\0'` or `L'\0'`. The size of a UTF-16 string literal is the total number of escape sequences, *universal-character-name*s, and other characters, plus one for each character requiring a surrogate pair, plus one for the terminating `u'\0'`. [*Note:* The size of a `char16_t` string literal is the number of code units, not the number of characters. — *end note*] [*Note:* Any *universal-character-name*s are required to correspond to a code point in the range [0,D800) or [E000,10FFFF] (hexadecimal) ([lex.charset]). — *end note*] The size of a narrow string literal is the total number of escape sequences and other characters, plus at least one for the multibyte encoding of each *universal-character-name*, plus one for the terminating `'\0'`.

Change in 5.13.5 [lex.string] paragraph 14:
Drafting note: Wording for string literal object initialization has been moved to a new paragraph.

Evaluating a *string-literal* results in a string literal object with static storage duration ([basic.stc])~~, initialized from the given characters as specified above~~. Whether all *string-literal*s are distinct (that is, are stored in nonoverlapping objects) and whether successive evaluations of a *string-literal* yield the same or a different object is unspecified. [*Note:* The effect of attempting to modify a *string-literal* is undefined. — *end note*]

Add a new paragraph (Z) after 5.13.5 [lex.string] paragraph 14:

String literal objects are initialized with the sequence of code unit values corresponding to the *string-literal*'s sequence of *s-char*s (for a non-raw string literal) and *r-char*s (for a raw string literal) in order as follows:

(Z.1) — The sequence of characters denoted by each contiguous sequence of *basic-s-char*s, *r-char*s, *simple-escape-sequence*s ([lex.ccon]), and *universal-character-name*s ([lex.charset]) is encoded to a code unit sequence using the *string-literal*'s associated character encoding. If a character lacks representation in the associated character encoding, then:

— If the *string-literal*'s *encoding-prefix* is absent or `L`, then the *string-literal* is conditionally-supported and an implementation defined code unit sequence is encoded.

— Otherwise, the *string-literal* is ill-formed.

When encoding a stateful character encoding, implementations should encode the first such sequence beginning with the initial encoding state and encode subsequent sequences beginning with the final encoding state of the prior sequence.

[*Note:* The encoded code unit sequence may differ from the sequence of code units that would be obtained by encoding each character independently. — *end note*]

(Z.2) — Each *numeric-escape-sequence* ([lex.ccon]) that specifies an integer value `V` contributes a single code unit with a value as follows:

— If `V` does not exceed the range of representable values of the *string-literal*'s code unit type, then the value is `V`.

— Otherwise, if the *string-literal*'s *encoding-prefix* is absent or `L`, and `V` does not exceed the range of representable values of the corresponding unsigned type for the underlying type of the *string-literal*'s code unit type, then the value is the unique value of the *string-literal*'s code unit type `T` that is congruent to `V` modulo 2^N, where N is the width of `T`.

— Otherwise, the *string-literal* is ill-formed.

When encoding a stateful character encoding, these sequences should have no effect on encoding state.

(Z.3) — Each *conditional-escape-sequence* ([lex.ccon]) contributes an implementation-defined code unit sequence.

When encoding a stateful character encoding, it is implementation-defined what effect these sequences have on encoding state.